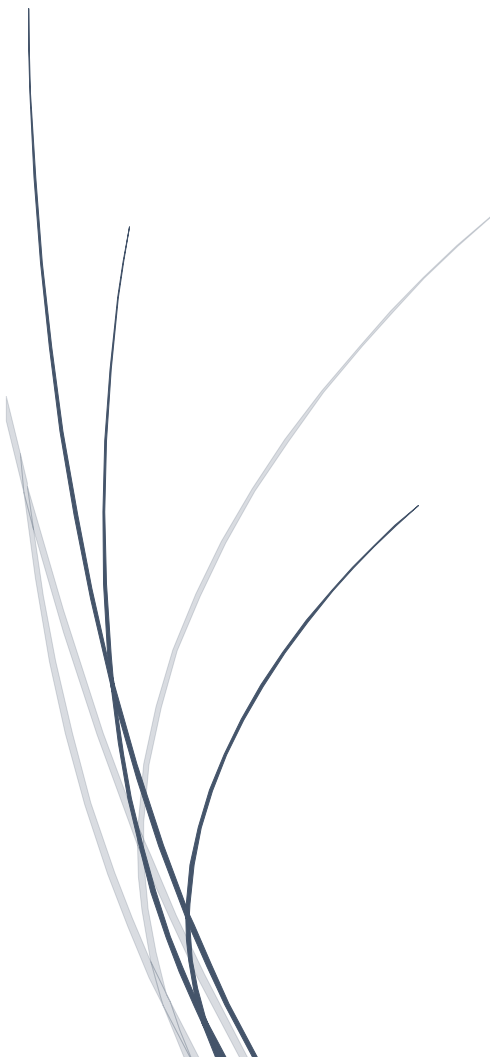


String in C++

Objectives

In this lab we will learn:

- C-String
- Declaration and Initialization of C String
- Comparison of C and C++ Strings
- String Object



1: C++ Strings

String is a collabtion of characters. There are two types of strings commonly used in C++ programming language:

C-strings (C-style Strings)

Strings that are objects of string class (The Standard C++ Library string class)

2: C-String

In C programming, the collaboration of characters is stored in the form of arrays.

- This is also supported in C++ programming.
- That is why it is called C-strings.
- C-strings are arrays of type char terminated with null character, that is, \0 (ASCII value of null character is 0).

2.1 Declaring and Initializing C-String

Example:

```
char myString[ ] = "Hello";
```

- In the above example, myString is a string
- myString holds 6 characters.
- Although,"Hello"has 5 characters, the null character \0 is added to the end of the string automatically.

We can write the strings in many different ways as given in the following examples

Example:

```
char myString[6] = "Hello";
```

```
char myString[ ] = {'H','e','l','l','o','\0'}
```

```
char myString[6] = {'H','e','l','l','o','\0'}
```

Unlike arrays, it is not necessary to use all the space allocated for the string. For example:

Example:

```
char myString[100] = "Hello";
```

Note:

You do not place the null character at the end of a string constant. The C++ compiler automatically places the `\0` at the end of the string when it initializes the array.

Example:

Ask the user to enter a string

```
#include  
  
using namespace std;  
  
int main() { char str[100]; //string type array of size 100  
  
cout << "Enter a string: ";  
  
cin >> str;           //Taking a string from user  
  
cout << "You entered: " << str << endl;  
  
cout << "Enter another string: ";  
  
cin >> str;           //Taking a string from user  
  
cout << "You entered: " << str << endl;  
  
return 0; }
```

Output:

Enter a string: Department You entered:

Department Enter another string: Electrical Engineering You entered: Electrical

Example:

C++ String to read a line of text

```
#include  
  
using namespace std;  
  
int main() {  
    char str[100];        //string type array of size 100  
  
    cout << "Enter a string: ";  
  
    cin.get(str, 100);    //taking strings from user using builtin function  
  
    cout << "You entered: " << str << endl;  
  
    return 0; }
```

Output:

Enter a string: Department You entered:

You Entered: Electrical Engineering You entered: Electrical

To read the text containing blank space, cin.get function can be used.

The cin.get function takes two arguments. First argument is the name of the string (address of first element of string) and second argument is the maximum size of the array.

In this program, str is the name of the string and 100 is the maximum size of the array.

3: String Object

- In C++, you can also create a string object for holding strings.
- Unlike using char arrays, string objects has no fixed length, and can be extended as per your requirement.

Example:

C++ String to read a line of text

```
#include
using namespace std;
int main() {
// Declaring a string object
string str;
cout << "Enter a string: ";
getline(cin, str); //taking strings from user using builtin function
cout << "You entered: " << str << endl;
return 0; }
```

Output:

Enter a string: Department You entered:

You Entered: Electrical Engineering You entered: Electrical

In this program, a string str is declared.

Then the string is asked from the user.

Instead of using cin >> or cin.get() function, you can get the entered line of text using getline().

The getline() function takes the input stream as the first parameter which is cin and str as the location of the line to be stored

4: Functions That Support Strings

C++ supports a wide range of functions that manipulate null-terminated strings.

Table 1: C++ String Function

String Function	Example	Description
strcpy()	strcpy(s1, s2);	Copies string s2 into string s1.
strcat()	strcat(s1, s2);	Concatenates string s2 onto the end of string s1.
strlen()	strlen(s);	Returns the length of string s.
strcmp()	strcmp(s1, s2);	Returns 0 if s1 and s2 are the same; less than 0 if s1 < s2; greater than 0 if s1 > s2.
strrev()	strrev(s);	Reverses the given string.
strlwr()	strlwr(s);	Converts string to lowercase.
strupr()	strupr(s);	Converts string to uppercase.

Lab Tasks:

Task 1: String Length Calculation

```
#include <iostream>                // Header file for input and output
#include <string>                   // Header file for string functions
using namespace std;              // Use standard namespace
int main()                        // Main function
{
    // Declare and initialize a string variable
    string message = "Welcome to C++ Programming";

    // Display the original string
    cout << "Message: " << message << endl;

    // Use length() function to calculate string length
    cout << "Length of string = " << message.length() << endl;

    return 0;}                   // End the program successfully
```

Task 2: String Concatenation

```
#include <iostream>           // Include input-output library
#include <string>              // Header file for string operations
using namespace std;         // Use standard namespace
int main() {                  // Main function
    // Declare and initialize first name
    string firstName = "Asad";

    // Declare and initialize last name
    string lastName = "Ullah";

    // Concatenate first name and last name with space
    string fullName = firstName + " " + lastName;

    // Display the full name
    cout << "Full Name: " << fullName << endl;
    return 0;                 // End program
}
```